ADOPT-99: Best Practice Summary

> **Series:** ADOPT — Observability Adoption & Maturity | **Notebook:** 99 | **Created:** March 2026 | **Last Updated:** 10/23/2026

Overview

This notebook consolidates every actionable best practice from the ADOPT series (notebooks 01 through 06) into definitive, categorized guidance. Each practice specifies exactly what to set, the priority level, and the source notebook. Use this as a checklist for platform maturity and adoption readiness. Numeric targets are community starting points, not Dynatrace-published thresholds — replace them with your own once you have a baseline.

Table of Contents

1. Agent Deployment and Host Monitoring
2. Data Ingestion and Grail Configuration
3. Alerting and Dynatrace Intelligence
4. SLOs and Success Metrics
5. Cost Optimization
6. Automation and Workflows
7. IAM and Access Control
8. Team Enablement and Organizational Readiness
9. Dashboards and Reporting
10. Configuration Management
11. Distributed Tracing and Spans
12. Business Observability
13. Coverage Audit and Staged Enablement

Prerequisites

Requirement	Details
Dynatrace Environment	Dynatrace SaaS (Grail). Not applicable to Dynatrace Managed, which has no Grail.
Permissions	`storage:logs:read`, `storage:spans:read`, `storage:metrics:read`, `storage:entities:read`, `storage:smartscape:read`, `storage:events:read`, `storage:system:read`, `storage:buckets:read`
Context	Familiarity with ADOPT-01 through ADOPT-06
Audience	Platform engineers, SREs, engineering managers, leadership

1. Agent Deployment and Host Monitoring

#	Best Practice	Recommended Setting/Value	Priority	Source
1.1	**OneAgent host coverage**	> 95% of all known hosts monitored (compare entity count against CMDB)	Critical	ADOPT-02
1.2	**Monitoring mode for application hosts**	Set to `FULL_STACK` for all hosts running application workloads	Critical	ADOPT-01, ADOPT-02
1.3	**Monitoring mode for non-application hosts**	Set to `INFRASTRUCTURE` for hosts that only need OS-level metrics (billed per host-hour instead of per GiB-hour of memory)	Recommended	ADOPT-05
1.4	**Agent version currency**	Every host on a OneAgent release still inside its support window (9 months Standard, 12 months Enterprise, from release); keep the number of distinct releases small	Critical	ADOPT-02, ADOPT-05

#	Best Practice	Recommended Setting/Value	Priority	Source
1.5	**Auto-update for non-production**	Enable OneAgent auto-update on all non-production hosts	Recommended	ADOPT-05
1.6	**Automated agent deployment**	Use cloud-init, Ansible, or equivalent to deploy OneAgent on every new host automatically	Recommended	ADOPT-05
1.7	**Host CPU/memory health check**	Run a weekly health query confirming that every host without a `paasVendorType` reports CPU and memory metrics; a missing series there is an agent issue. PaaS hosts (Kubernetes application-only, Fargate) report no host metrics, so check them through their workloads instead	Recommended	ADOPT-02
1.8	**Read monitoring mode from billing**	Use the billing usage events, not the classic `monitoringMode` property, which is empty for many hosts	Recommended	ADOPT-02, ADOPT-06

2. Data Ingestion and Grail Configuration

#	Best Practice	Recommended Setting/Value	Priority	Source
2.1	**Multi-signal ingestion**	Ingest all 5 telemetry types: metrics, logs, spans, events, and business events	Critical	ADOPT-01
2.2	**Log ingestion stability**	Daily variance < 10%; monitor billed log ingest (GiB per day) over 7 days	Recommended	ADOPT-02
2.3	**Span ingestion active**	Confirm > 0 spans per hour at all times; a zero reading means broken instrumentation	Critical	ADOPT-02

#	Best Practice	Recommended Setting/Value	Priority	Source
2.4	**Grail bucket naming convention**	`<team>_<datatype>_<env>` (e.g., `checkout_logs_prod`)	Recommended	ADOPT-05
2.5	**Bucket-specific retention**	High-value logs: 35+ days. Medium-value: 7-14 days. Debug/verbose: drop at ingest, or 7 days	Recommended	ADOPT-05
2.6	**OpenPipeline log routing**	Route logs to the correct bucket by source using OpenPipeline rules; never leave all data in `default_logs`	Recommended	ADOPT-05
2.7	**Drop DEBUG/TRACE in production**	Configure an OpenPipeline rule to discard `DEBUG` and `TRACE` records from production sources	Recommended	ADOPT-05
2.8	**Use bucket targeting in queries**	Specify `bucket:{"name"}` in recurring DQL fetch commands when you know which bucket holds the data, to reduce scanned bytes	Optional	ORGNZ

3. Alerting and Dynatrace Intelligence

#	Best Practice	Recommended Setting/Value	Priority	Source
3.1	**Alert quality**	Track the share of problems that close within 5 minutes and the problems that recur on one component; both should trend down. Do not use the frequent-issue flag — frequent issue detection is being phased out	Critical	ADOPT-01, ADOPT-03
3.2	**Dynatrace Intelligence enabled**	Dynatrace Intelligence problem detection must be active; verify by confirming problems are detected in the last 7 days	Critical	ADOPT-01, ADOPT-02

#	Best Practice	Recommended Setting/Value	Priority	Source
3.3	**Keep transient problems away from people**	For problem types that consistently resolve within 5 minutes, set a Minimum duration on the notifying problem-triggered workflow (FAQ-21), or tune the detector	Recommended	ADOPT-05
3.4	**Maintenance windows**	Configure maintenance windows for all planned change periods; a high share of problems raised during maintenance is a prompt to check detection sensitivity	Recommended	ADOPT-03, ADOPT-05
3.5	**Alert ownership**	Route every problem to an owning team through ownership tags and problem-triggered workflows; no unowned alerts (FAQ-21)	Recommended	ADOPT-05
3.6	**Monthly alert review**	Review the top 10 problem types by volume and short-lived share monthly; tune detectors or set a Minimum duration on notifications	Recommended	ADOPT-05
3.7	**Filter duplicate problems from metrics**	Exclude `dt.davis.is_duplicate == true` when calculating MTTD and problem duration	Critical	ADOPT-03
3.8	**Exclude maintenance from MTTR**	Always exclude `maintenance.is_under_maintenance == true` when calculating MTTR	Recommended	ADOPT-03

4. SLOs and Success Metrics

#	Best Practice	Recommended Setting/Value	Priority	Source
4.1	**Define SLOs for top 10 services**	Set availability and latency SLOs for the 10 most business-critical services	Critical	ADOPT-05

#	Best Practice	Recommended Setting/Value	Priority	Source
4.2	**SLO burn-rate alerting**	Build burn-rate alerting for every SLO as described in SLO-04; static thresholds are insufficient	Critical	ADOPT-01, ADOPT-05
4.3	**Track MTTD**	Target: median under 5 minutes. Measure weekly as the gap between `event.start` and the problem's `CREATED` transition on `fetch events`	Critical	ADOPT-03
4.4	**Track problem duration (MTTR)**	Set the target from your own 30-day baseline; measure weekly and trend by category. It measures how long a problem stayed open, not DORA's failed deployment recovery time	Critical	ADOPT-03
4.5	**Track weekly problem count**	Trend must be flat or declining; an increasing trend signals environmental degradation	Recommended	ADOPT-03
4.6	**Track change failure rate**	Send deployment events, then measure the share of deployments followed within an hour by a problem on the same entity; compare with current DORA benchmarks	Recommended	ADOPT-03
4.7	**Establish 30-day baselines**	Record current MTTD, problem duration, short-lived problem share, problem count, and CFR using a 30-day window before setting targets	Critical	ADOPT-03
4.8	**Re-baseline quarterly**	Recalculate baselines every 90 days to account for growth and environmental changes	Recommended	ADOPT-03
4.9	**Set 3-month and 6-month targets**	Define explicit improvement targets for each metric at 3-month and 6-month horizons	Recommended	ADOPT-03

5. Cost Optimization

#	Best Practice	Recommended Setting/Value	Priority	Source
5.1	**Audit high-volume log sources**	Identify top 15 log sources by volume weekly; evaluate each for filtering, sampling, or reduced retention	Recommended	ADOPT-05
5.2	**Log retention tiers**	High-value logs: 35+ days. Standard logs: 7-14 days. Debug/verbose: drop at ingest, or 7 days	Recommended	ADOPT-05
5.3	**Span sampling for high-volume services**	Low-value spans: sample at 10:1. Medium-value: 2:1. High-value: full retention	Recommended	ADOPT-05
5.4	**Use `samplingRatio` for exploratory queries**	Set `samplingRatio:10` or `samplingRatio:100` on ad-hoc `fetch logs` queries against large datasets; multiply counts back up	Optional	OPLOGS-99
5.5	**Bound expensive queries with `scanLimitGBytes`**	`scanLimitGBytes:100` stops one query after 100 GB (whole GB only; the default is 500). A stopped query returns **partial results** with a `SCAN_LIMIT_GBYTES` warning, so treat it as a per-query safety stop, not a cost budget. Never set `-1` (no limit) on a recurring query	Optional	ORGNZ-02
5.6	**Replace raw log storage with metric extraction**	For log patterns you only need to count (not inspect), extract as metrics via OpenPipeline instead of storing raw records	Optional	ADOPT-05

#	Best Practice	Recommended Setting/Value	Priority	Source
5.7	**Review monitoring mode quarterly**	Verify from the billing events that Full-Stack hosts genuinely need Full-Stack; move hosts that run nothing you trace to Infrastructure	Recommended	ADOPT-05, ADOPT-06
5.8	**Track daily log ingestion over 7 days**	Run the 7-day billed-log-ingest query monthly; flag any day exceeding 2x the baseline as a cost anomaly	Recommended	ADOPT-02, ADOPT-05

6. Automation and Workflows

#	Best Practice	Recommended Setting/Value	Priority	Source
6.1	**Automate alert triage and routing**	Create a Dynatrace Workflow that routes detected problems to the correct team channel; run daily	Critical	ADOPT-05
6.2	**Automate top 5 frequent problem responses**	Build Workflows for the 5 most frequent, well-understood problem types; start with triage, progress to remediation	Recommended	ADOPT-05
6.3	**Minimum duration for transient problems**	Set a 5-minute Minimum duration on the problem-triggered workflows that notify people, for problem types that auto-resolve within 5 minutes (FAQ-21)	Recommended	ADOPT-05
6.4	**Scheduled DQL reports for leadership**	Create a weekly scheduled notebook that runs MTTR, MTTD, noise ratio, and problem count queries automatically	Recommended	ADOPT-05

#	Best Practice	Recommended Setting/Value	Priority	Source
6.5	**Automated postmortem data collection**	Build a Workflow triggered by problem closure that collects timeline, root cause, affected entities, and duration into a structured record	Optional	ADOPT-05
6.6	**Dynatrace Intelligence forecasting for capacity**	Enable Dynatrace Intelligence metric forecasting for CPU, memory, and disk on all production hosts; review monthly	Optional	ADOPT-01, ADOPT-05

7. IAM and Access Control

#	Best Practice	Recommended Setting/Value	Priority	Source
7.1	**Eliminate shared admin accounts**	Replace shared admin credentials with individual user accounts bound to IAM groups	Critical	ADOPT-05
7.2	**Least-privilege IAM policies**	Define policies per team granting only the permissions required for their role	Critical	ADOPT-05
7.3	**Data-scoped access**	Restrict data access with security context and policy boundaries (ORGNZ, IAM). Segments only filter what a user sees on screen — they are not access control (FAQ-24). Management zones remain the classic equivalent	Recommended	ADOPT-05
7.4	**IAM group per team**	Create an IAM group for every team that uses Dynatrace (e.g., `sre-team`, `platform-team`, `app-dev`)	Recommended	ADOPT-05

8. Team Enablement and Organizational Readiness

#	Best Practice	Recommended Setting/Value	Priority	Source
8.1	**1 Dynatrace champion per team**	Identify and train at least 1 internal champion per engineering team; complete the platform engineer track within 6 weeks	Critical	ADOPT-04
8.2	**2 DQL-literate members per team**	Every team must have at least 2 people who can write `fetch`, `filter`, `summarize` queries independently	Critical	ADOPT-04
8.3	**100% teams with self-service dashboards**	Every team creates and maintains their own service dashboard within 6 months	Recommended	ADOPT-04
8.4	**12-week enablement program**	Run a structured 12-week program: Weeks 1-2 orientation, 3-4 DQL, 5-6 role-specific, 7-8 dashboards, 9-10 incident practice, 11-12 champion certification	Recommended	ADOPT-04
8.5	**Reduce escalation tickets by 50%**	Track escalation tickets to the platform team; target 50% reduction within 3 months of champion program launch	Recommended	ADOPT-04
8.6	**Monthly champion sync**	Hold a monthly meeting for all Dynatrace champions to share learnings and review platform changes	Recommended	ADOPT-04
8.7	**Quarterly skills refresh**	Run a quarterly training session for champions covering new features and advanced patterns	Optional	ADOPT-04

#	Best Practice	Recommended Setting/Value	Priority	Source
8.8	**Role-based learning paths**	Assign the notebook study sequence per role: SRE (ONBRD, OPLOGS, SPANS, ALERT, WFLOW, SLO, SYNTH, AIOPS), Developer (ONBRD 1-5, SPANS, OTEL, ORGNZ 1-3, FAQ-03/15), Platform (ONBRD, K8S, IAM, AUTOM, ORGNZ, OPMIG, FINOPS), Manager (ADOPT, ONBRD 1-3, ORGNZ 1, ALERT-01/SLO-01, FINOPS-01)	Recommended	ADOPT-04

9. Dashboards and Reporting

#	Best Practice	Recommended Setting/Value	Priority	Source
9.1	**3 standard dashboard templates**	Create exactly 3 organizational templates: Infrastructure Health, Application Health, Business KPIs	Recommended	ADOPT-05
9.2	**Retire unused dashboards**	Audit dashboards quarterly; delete any dashboard with no views in the last 90 days	Optional	ADOPT-05
9.3	**Weekly platform health scorecard**	Review 8 metrics weekly: host coverage, agent version currency, service discovery, log ingestion stability, span ingestion, ActiveGate health, Dynatrace Intelligence activity, alert quality	Critical	ADOPT-02
9.4	**Scorecard thresholds**	8/8 green = Healthy. 6-7 = Minor gaps (fix this sprint). 4-5 = Significant (prioritize). < 4 = Critical (escalate)	Critical	ADOPT-02
9.5	**Monthly ROI report**	Combine MTTR/MTTD trends, problem count trends, data volume, enablement progress, and estimated cost avoidance into a single monthly report for leadership	Recommended	ADOPT-05

#	Best Practice	Recommended Setting/Value	Priority	Source
9.6	**SLO dashboard for leadership**	Create a dedicated SLO dashboard showing burn rate and compliance for the top 10 services	Recommended	ADOPT-05

10. Configuration Management

#	Best Practice	Recommended Setting/Value	Priority	Source
10.1	**Configuration as code**	Export Dynatrace configuration with Monaco or the Terraform provider and store it in a Git repository	Recommended	ADOPT-01, ADOPT-05
10.2	**CI/CD pipeline for config deployment**	Deploy Dynatrace configuration changes through a CI/CD pipeline; never apply changes manually in production	Recommended	ADOPT-05
10.3	**Drift detection**	Check weekly for manual configuration changes — Terraform `plan` reports drift natively; with Monaco, download the live configuration on a schedule and diff it against the repository	Recommended	ADOPT-05
10.4	**Dashboard-as-code templates**	Store dashboard JSON templates in the config repo; provision new service dashboards from templates via Monaco or Terraform	Optional	ADOPT-05

11. Distributed Tracing and Spans

#	Best Practice	Recommended Setting/Value	Priority	Source
11.1	**OpenTelemetry for uninstrumented services**	Implement OpenTelemetry SDKs for any service not covered by OneAgent auto-instrumentation	Recommended	ADOPT-05
11.2	**OTLP endpoint routing**	Send OTel data to the Dynatrace OTLP endpoint; add an OpenTelemetry Collector when you need batching, sampling, or fan-out (OTEL)	Recommended	ADOPT-05
11.3	**Validate trace context propagation**	Verify end-to-end trace context propagation across all service boundaries; broken propagation = blind spots	Critical	ADOPT-05
11.4	**Span volume monitoring**	Query top 10 services by span count weekly; apply head-based or tail-based sampling to high-volume, low-value services	Recommended	ADOPT-05

12. Business Observability

#	Best Practice	Recommended Setting/Value	Priority	Source
12.1	**Instrument 3-5 key business transactions**	Identify the 3-5 highest-value business transactions (e.g., checkout, login, search) and emit business events for each	Recommended	ADOPT-05
12.2	**Business-context alerting**	Create alerts that trigger on business metric thresholds (e.g., revenue drop, conversion rate decline), not just technical metrics	Recommended	ADOPT-05

#	Best Practice	Recommended Setting/Value	Priority	Source
12.3	**Correlate technical health with business KPIs**	Build dashboards that show service latency/error rate alongside business metrics (conversion, revenue) on the same view	Optional	ADOPT-05
12.4	**Entity ownership assignment**	Assign an owning team to every monitored service and application entity in Dynatrace	Recommended	ADOPT-01, ADOPT-04

13. Coverage Audit and Staged Enablement

#	Best Practice	Recommended Setting/Value	Priority	Source
13.1	**Run a Wave-0 coverage audit**	Measure hosts by monitoring mode, log coverage, consumption by mode, and deep-monitoring exceptions; record as the enablement baseline	Critical	ADOPT-06
13.2	**Segment the estate before enabling**	Match mode to segment: revenue-path apps → Full-Stack, backing services → Infrastructure, long-tail fleet → Discovery	Critical	ADOPT-06
13.3	**Enable in connected segments, not layers**	Bring one application's full path (Full-Stack + logs + RUM) online before spreading a single layer estate-wide	Recommended	ADOPT-06
13.4	**Sequence Waves 1–4 in order**	Core apps to Full-Stack → logs → RUM/synthetic → AppSec and optimization; each wave rides the modules the prior wave already enabled	Recommended	ADOPT-06

#	Best Practice	Recommended Setting/Value	Priority	Source
13.5	**Gate each wave on a moved metric**	Use ADOPT-03's success metrics as value gates (MTTR delta, log-evidence rate, user-impact rate, vulnerability-backlog trend) before funding the next wave	Critical	ADOPT-06
13.6	**Re-run the Wave-0 audit at every gate**	Coverage decays between waves as new hosts land in default modes and exclusion rules outlive their incidents; re-audit quarterly	Recommended	ADOPT-06

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.